

I. Об'єктно-орієнтоване програмування (ООП) та UML-моделювання

1. Парадигма ООП, класи та об'єкти:

- Дайте визначення ООП як парадигми програмування.
- Поясніть різницю між класом та об'єктом, використовуючи поняття «креслення» та «конкретного екземпляра».
- Для яких сутностей визначено шаблони поведінки, а для яких — стан?

2. Основні концепції ООП: Розкрийте зміст чотирьох наріжних концепцій парадигми:

- *Абстракція*: визначення концептуальних меж об'єкта та відкидання незначних деталей у заданому контексті.
- *Інкапсуляція*: механізм об'єднання коду і даних, приховування деталей реалізації та захист внутрішнього стану від непередбачених змін ззовні.
- *Наслідування*: створення класу на основі існуючого. Чим розплачується розробник за використання спадкування у контексті інтерфейсу батьківського класу? Які обмеження існують щодо кількості «батьків» та реалізації інтерфейсів?
- *Поліморфізм*: використання спільного інтерфейсу для обробки різних спеціалізованих типів.

3. UML та діаграма класів:

- Що таке діаграма класів у контексті статичного зображення структури моделі?
- Які області видимості позначаються символами + та - на діаграмах?

4. Відношення (зв'язки) між класами в UML: Опишіть логіку, графічне посилання та кодову реалізацію зв'язків на основі «загальної картини»:

- *Залежність*: у яких випадках зміни в одному класі торкаються іншого та де саме в коді (конструктори, типи параметрів методів) з'являється цей зв'язок?
- *Асоціація*: як відображаються структурні відношення взаємодії, при яких об'єкти посиляються один на одного, але залишаються незалежними? Чому асоціацію вважають суворішим варіантом залежності?
- *Агрегація проти Композиції*: порівняйте ці два різновиди відношення між цілим і його частинами. Як вони відрізняються за критерієм керування життєвим циклом компонентів (на прикладі ліквідації університету/факультетів/студентів або взаємодії машини з водієм та колесом)? Якими ромбами вони позначаються на діаграмах?
- *Наслідування та реалізація*: особливості відображення зв'язків типу «є» (is) за допомогою суцільних/пунктирних ліній та трикутних закінчень. Чому при будь-якому з цих зв'язків клас А все одно залежить від класу В?

II. Базові принципи проектування та SOLID

5. Метрики комерційного ПЗ та проблеми повторного використання:

- Які дві метрики є найважливішими при розробці комерційних продуктів та як повторне використання архітектури допомагає їх знизити?
- Назвіть три основні проблеми старого коду, які заважають пристосувати його до нових умов.

6. Універсальні принципи проектування:

- *Інкапсулюйте те, що змінюється*: поясніть суть відокремлення нестабільних аспектів програми від постійних. Проілюструйте інкапсуляцію на рівні методу та на рівні класу на прикладі розрахунку фінальної суми замовлення й винесення податкової логіки в TaxCalculator.
- *Програмуйте на рівні інтерфейсу, а не реалізації*: чому код повинен залежати від абстракцій? Опишіть 4 кроки переходу до інтерфейсної залежності на прикладі взаємодії класів Cat та Sausage / Food.
- *Віддавайте перевагу композиції перед спадкуванням*: які п'ять основних проблем має наслідування (порушення інкапсуляції, розростання ієрархії тощо)? Проаналізуйте це на прикладі ієрархії транспортних засобів (Car/Truck, Electric/Combustion, Autopilot/Manual). Як делегування через композицію дозволяє динамічно замінювати поведінку (наприклад, тип двигуна) під час виконання програми?

7. Принципи SOLID: Яка головна мета цієї аббревіатури? Розкрийте суть кожного принципу на основі лекційних прикладів:

- *Single Responsibility Principle (SRP)*: принцип єдиного обов'язку на прикладі розділення функціоналу класів Employee та TimeSheetReport.
- *Open/Closed Principle (OCP)*: що означає відкритість для розширення та закритість для змін? Як уникнути ламання існуючого коду у класі Order при додаванні нових типів доставки за допомогою інтерфейсу Shipping (Ground/Air)?
- *Liskov Substitution Principle (LSP)*: чому підкласи повинні доповнювати, а не підміняти поведінку базового класу? Детально поясніть формальні вимоги до перевизначених методів (типи параметрів, типи значень, що повертаються, виключення, передумови, післяумови, інваріанти) на прикладах методів годування котів feed(Cat) / feed(Animal) / feed(BengalCat) та методу buyCat().
- *Interface Segregation Principle (ISP)*: чому багато спеціалізованих інтерфейсів краще за один універсальний? Опишіть проблему «широкого інтерфейсу» на прикладі проектування бібліотеки CloudProvider для повнофункціонального Amazon та обмеженого Dropbox.
- *Dependency Inversion Principle (DIP)*: чому класи високого рівня (бізнес-логіка) не повинні залежати від класів низького рівня (базові операції)? Як створення інтерфейсу змінює напрямок проектування на протилежний (на прикладі зв'язку BudgetReport та MySQLDatabase / Database)?

III. Патерни (шаблони) проектування

8. Поняття патерна:

- Що таке патерн проектування та які три спільні ознаки він має?
- Розкрийте суть формули: Патерн = Контекст + Проблема + Рішення.

9. Рівні абстракції в проектуванні: Порівняйте три рівні рішень за масштабом, специфікою та архітектурною аналогією з будівництвом (забивання цвяха, типовоє вікно, проект усього будинку):

- *Ідіоми (нижній рівень)*: прив'язка до стилю мови програмування (приклади з Kotlin та Python).
- *Шаблони проектування (середній рівень)*: універсальна тактика взаємодії кількох класів в ООП (наприклад, Strategy, Observer).
- *Архітектурні патерни (верхній рівень)*: стратегічний каркас та визначення життєвого циклу всього додатка (наприклад, MVC, Microservices).

10. Специфіка патернів (Pattern vs Code / Pattern vs Algorithm):

- Чому патерн не є конкретним кодом, який можна просто скопіювати у програму?
- Протиставте патерн алгоритму через критерії «що робити» та «як організувати», використовуючи аналогію з кулінарним рецептом та проектом кухні.

11. Структура, класифікація та історія:

- З яких шести обов'язкових елементів складається опис будь-якого патерна?
- Опишіть класифікацію патернів за призначенням: породжуючі, структурні та поведінкові.
- Хто є автором концепції шаблонів, як ідеї перейшли у розробку графічних оболонок на Smalltalk та C++ і яка роль «Банди чотирьох» (GoF) та їхньої книги 1994 року у прориві цієї методики? Назвіть три причини, навіщо розробнику знати патерни.

IV. Породжуючі шаблони та концепція «Фабрик»

12. Породжуючі шаблони: Яку роль виконують твірні шаблони у процесі побудови об'єктів та від яких проблем безпеки й зайвих залежностей вони захищають?

Перерахуйте п'ять класичних твірних шаблонів.

13. Термінологія «Фабрик»: Щоб уникнути плутанини під час вивчення матеріалу, чітко розмежуйте чотири поняття, які програмісти часто називають одним словом «Фабрика»:

- Загальна концепція проектування «Фабрика».
- *Створюючий метод*: проста функція-обгортка над конструктором продукту.
- *Статичний фабричний метод*: статична варіація, що виступає альтернативним конструктором свого ж класу.
- *Патерн Проста фабрика (Simple Factory)*: окремий клас, що містить метод із великим умовним оператором, який динамічно вирішує, який саме екземпляр продукту створити на основі вхідного параметра (на прикладі фабрики різних типів автомобілів).

5. Фабричний метод (Factory Method): Суть, структура та діаграма класів. Чим фабричний метод відрізняється від «простої фабрики» (Simple Factory)?

6. Абстрактна фабрика (Abstract Factory): Проблема взаємозамінності сімейств продуктів. Опишіть архітектуру кросплатформного інтерфейсу (наприклад, GUI для різних ОС) за допомогою цього шаблону.

7. Будівельник (Builder): Покрокове конструювання складних об'єктів. Роль Директора (Director). Реалізація Fluent API в Java за допомогою внутрішніх класів.

8. Порівняльний аналіз: у яких ситуаціях варто обрати **Абстрактну фабрику**, а в яких — **Будівельник**?
9. **Прототип (Prototype)**: Клонування об'єктів без прив'язки до їхніх конкретних класів. Особливості реалізації глибокого (deep copy) та поверхневого (shallow copy) копіювання в Java (інтерфейс Cloneable, перевизначення методу clone(), копіювальні конструктори).
10. **Одинак (Singleton)**: Мета та застереження щодо використання. Чому Singleton часто називають антипатерном? Проблеми з модульним тестуванням (Unit Testing) об'єктів-одинаків.

III. Структурні шаблони (Structural Patterns)

12. **Адаптер (Adapter)**: Забезпечення сумісності несумісних інтерфейсів. Порівняйте дві реалізації: Адаптер об'єктів (через композицію) та Адаптер класів (через множинне успадкування/інтерфейси в Java).
13. **Міст (Bridge)**: Розділення абстракції та реалізації так, щоб вони могли змінюватися незалежно. Як шаблон дозволяє уникнути комбінаторного вибуху кількості підкласів під час розширення функціоналу?
14. **Компонувальник (Composite)**: Побудова деревоподібних структур об'єктів (ціле/частина). Реалізація однакового інтерфейсу для елементарних об'єктів (листя) та контейнерів.
15. **Декоратор (Decorator)**: Динамічне розширення функціональності об'єкта за допомогою композиції. Наведіть приклади Декораторів у стандартній бібліотеці java.io (наприклад, BufferedReader, FileInputStream).
16. **Фасад (Facade)**: Спрощений інтерфейс до складної підсистеми класів чи бібліотек. Чим Фасад принципово відрізняється від Медіатора чи Адаптера?
17. **Легковаговик (Flyweight)**: Оптимізація пам'яті за рахунок поділу стану об'єкта на внутрішній (intrinsic) та зовнішній (extrinsic). Наведіть приклади реалізації пулів об'єктів у Java (String Pool, кешування в Integer.valueOf()).
18. **Замісник (Proxy)**: Контроль доступу до об'єкта. Види проксі (віртуальний, захисний, логуючий, кешуючий). Поняття та механізм роботи Dynamic Proxy в Java за допомогою java.lang.reflect.Proxy.

IV. Поведінкові шаблони (Behavioral Patterns)

19. **Ланцюжок відповідальності (Chain of Responsibility)**: Передача запитів по ланцюжку обробників. Приклади використання (сервлети/фільтри в веб-застосунках, системи логування чи валідації).
20. **Команда (Command)**: Перетворення запиту на окремий об'єкт. Реалізація механізму скасування/повторення операцій (Undo/Redo) та черг завдань за допомогою Команди.
21. **Ітератор (Iterator)**: Послідовний обхід елементів колекції без розкриття її внутрішньої структури. Особливості реалізації інтерфейсів Iterator та Iterable у Java.
22. **Посередник (Mediator)**: Зниження взаємозалежності безлічі класів через створення єдиної точки комунікації. Опишіть роботу шаблону на прикладі взаємодії елементів форми інтерфейсу користувача (UI).

23. **Знімок (Memento):** Фіксація та відновлення внутрішнього стану об'єкта без порушення інкапсуляції. Роль класів-учасників: Originator, Memento, Caretaker.
24. **Спостерігач (Observer):** Механізм підписки типу «один до багатьох». Реалізація архітектури, керованої подіями (Event-driven). Роль Спостерігача у шаблонах MVC / MVVM.
25. **Стан (State):** Зміна поведінки об'єкта залежно від його внутрішнього стану. Порівняйте шаблон Стан із кінцевими автоматами (FSM). Як позбутися багаторівневих блоків switch-case чи if-else за допомогою цього шаблону?
26. **Стратегія (Strategy):** Визначення сімейства алгоритмів та їхня взаємозамінність у процесі виконання (runtime). Порівняйте шаблони **Стратегія** та **Стан**: у чому схожість їхніх структур і принципова різниця в інтенції?
27. **Шаблонний метод (Template Method):** Визначення скелета алгоритму в базовому класі з делегуванням конкретних кроків підкласам. Порівняння із шаблоном Стратегія (принцип «успадкування проти композиції»).
28. **Відвідувач (Visitor):** Додавання нових операцій до існуючої структури класів без їхньої модифікації. Поняття механізму подвійного диспетчеризування (Double Dispatch) та його емуляція в Java за допомогою методів accept() та visit().